

# seL4 docker build environment

## Page Content

- [General](#)
- [Setup Docker Environment](#)
- [Docker Environments](#)
- [Container Internals and Maintenance](#)
- [Pitfalls](#)
- [Building with CMake](#)

## General

- this page covers the seL4 containers
- see [Docker Registry](#) for our internal containers in the in the [Trentos M-SDK](#)
- see [Docker](#) for generic installation on the host OS and how it works.

## Setup Docker Environment

- seL4 build environment:
  - GIT repos with scripts to start it: <https://research.csiro.au/tsblog/getting-started-sel4-camkes-l4v-dependencies>
  - containers: <https://store.docker.com/profiles/trustworthysystems>
  - the whole docker setup is just one makefile that sets up things and runs the containers, basically follow the description at <https://github.com/SEL4PROJ/seL4-CAMkes-L4v-dockerfiles>, it also explains a lot more how thing work, which tricks are used and why security implication this might have on a system with multiple users. Below is just a brief description "user\_sel4".
- setup:
  - clone the repo with the scripts

```
# clone the repo into a sub-folder sel4-CAMkes-L4v-dockerfiles
git clone https://github.com/SEL4PROJ/seL4-CAMkes-L4v-dockerfiles
```

- Some of the internal projects (seos\_tests, seos\_key\_store...) require additional tools for the build process (doxygen, cppcheck), that are not available in the official repository mentioned above by default. Instead of cloning the official GitHub repo, use our copy at <https://bitbucket.hensoldt-cyber.systems/projects/EXT/repos/sel4-camkes-l4v-dockerfiles> and the branch "hensoldt-cyber":

```
# clone the repo into a sub-folder sel4-CAMkes-L4v-dockerfiles
git clone -b hensoldt-cyber ssh://git@bitbucket.hensoldt-cyber.systems:7999/ext/sel4-camkes-l4v-dockerfiles.git
```

## Docker Environments

- all build are executed inside the container it contains the whole toolchain.
- inside docker, the folder /host is a shared folder with the host. The env variable HOST\_DIR specifies where the shared workspace is, it must be set when starting the container
- inside the container a user will be created that has the same GID/UID as the current user on the host (so one does not run as docker' default "root"). Advantage is, that all files created from anything running in the container can be accessed easily outside the container, as they are owned by the current user
- the docker container is started via a makefile

```
# start the container with the current directory mounted at /host inside the container.
make -C <path-to-docker-repo> <target> HOST_DIR=$(pwd)
```

- make targets

| Target     | Container action |     | Toolchains |     |        |        | Notes                                                                                                   |
|------------|------------------|-----|------------|-----|--------|--------|---------------------------------------------------------------------------------------------------------|
|            | build            | run | Intel      | ARM | RISC-V | CAMkes |                                                                                                         |
| sel4       | ✓                | ✓   | ✓          | ✓   | ✗      | ✗      | build all containers from scratch. Usually, you don't want to do this, but use the pre-build containers |
| sel4-riscv | ✓                | ✓   | ✓          | ✓   | ✓      | ✗      |                                                                                                         |

|                   |   |   |   |   |   |   |                                                    |
|-------------------|---|---|---|---|---|---|----------------------------------------------------|
| user_sel4         | ✗ | ✓ | ✓ | ✓ | ✗ | ✗ | use this for bare metal seL4 development           |
| user_sel4-riscv   | ✗ | ✓ | ✓ | ✓ | ✓ | ✗ | use this for bare metal seL4 development on RISC-V |
| camkes            | ✓ | ✓ | ✓ | ✓ | ✗ | ✓ |                                                    |
| camkes-riscv      | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |                                                    |
| user_camkes       | ✗ | ✓ | ✓ | ✓ | ✗ | ✓ | use this for CAmkES development                    |
| user_camkes-riscv | ✗ | ✓ | ✓ | ✓ | ✓ | ✓ | use this for CAmkES development on RISC-V          |

## Container Internals and Maintenance

- the RISC-V toolchain is not part of the standard "[user\_]seL4" docker environment yet, it's implemented as another layer on top.
- update the pre-build docker containers images from <https://hub.docker.com/u/trustworthysystems>

```
docker pull trustworthysystems/seL4
docker pull trustworthysystems/camkes-riscv
docker pull trustworthysystems/camkes
```

## Pitfalls

- all absolute paths inside docker containers are different to these on host OS.
  - The shared folder is usually something from the home directory (e.g. "home/jondoe01/myProject"), and mapped at "/host" in the container.
  - Some script may work differently or fail when invoked on the host OS or in the container.
  - When CMake creates a build environment, it uses absolute paths for the generated build scripts - thus invoking them outside the container is likely to fail.

## Building with CMake

- our repos should provide a "build.sh" script. So the usual flow will be:

```
mkdir someProject
git clone https://some.thing/repo.git src
src/build.sh -DSOME_PARAMETER=42
```

- native seL4 repos provide a "init-build.sh" script to set up the CMake build environment.
  - the usual flow will be:

```
mkdir someProject
git clone https://some.thing/repo.git
mkdir build
cd build
../something/init-build.sh -DSOME_PARAMETER=42
```

- Internally, "init-build.sh" basically invokes "cmake" and create ninja build script, where ninja is basically a lightweight and much faster alternative of make (see <https://ninja-build.org>). So there is something inside like:

```
cmake -G Ninja ....
```

- the actual build is invoked with

```
cd build
ninja
# Verbose output: ninja -v
```

- For the tutorials
  - the setup parameters are

```

# seL4 Tutorials:
# $ ../init-build.sh \
#   -DTUTORIAL=<PREFERENCE> \
#   -DTUT_BOARD=<PREFERENCE> \
#   -DTUT_ARCH=<PREFERENCE> \
#   -DBUILD_SOLUTIONS=<BOOLEAN_PREFERENCE>
#
# TUT_BOARD=pc, TUT_ARCH=x86: This yields an x86-pc build.
# TUT_BOARD=pc, TUT_ARCH=x86_64: This yields an x86_64-pc build.
# TUT_BOARD=zynq7000 -DAARCH32=1: This yields an armv7-zynq7000 build.

../init-build.sh -DTUTORIAL=hello-2 -DTUT_BOARD=spike -DRISCV64=1 -DLibUtilsDefaultZfLogLevel=2
# ../init-build.sh -DTUTORIAL=hello-2 -DTUT_BOARD=zynq7000 -DAARCH32=1 -
DLibUtilsDefaultZfLogLevel=2

# CAMkES Tutorials:
# $ ../init-build.sh -DCAMKES_APP=addder

```

- the build also creates a script to invoke an emulator. It used to be a shell script, but now it's a python script to support a much more flexible invocation of the simulator.

```

# run a simulator for platform zynq7000
# $ qemu-system-arm -nographic -M xilinx-zynq-a9 -nographic -m size=1024M -s -serial null -
serial mon:stdio -kernel ../images/hello-2-image-arm-zynq7000
#
simulate

```